

## Recherche dichotomique - *Binary search*

---

# 1. Introduction



## 1.1 Exemple

Pour introduire la recherche dichotomique, nous allons considérer l'exemple suivant.

Soit  $A$  un tableau de  $n$  valeurs **trié**, on cherche à vérifier si un élément  $a$  est présent dans ce tableau.

## 1.1 Exemple

Pour introduire la recherche dichotomique, nous allons considérer l'exemple suivant.

Soit  $A$  un tableau de  $n$  valeurs **trié**, on cherche à vérifier si un élément  $a$  est présent dans ce tableau.

Il suffit de considérer le fait que  $A$  soit trié. Pour faire usage de cette information, nous nous plaçons sur l'élément  $A[\frac{n}{2}]$  et nous constatons 3 cas :

- $A[\frac{n}{2}] < a$ , donc  $a$  peut uniquement être après l'indice  $\frac{n}{2}$  (soit dans  $A[\frac{n}{2} + 1, n]$ )
- $A[\frac{n}{2}] > a$ , donc  $a$  peut uniquement être avant l'indice  $\frac{n}{2}$  (soit dans  $A[1, \frac{n}{2} - 1]$ )
- $A[\frac{n}{2}] = a$ , donc  $a \in A$ .

Donc si nous ne trouvons pas la valeur, on divise par 2 l'espace de recherche, et on applique l'algorithme à nouveau sur la bonne partie de  $A$ .

## 1.2 Implémentation de l'exemple

```
def bin_search(A : list[int], x : int):  
    # l et r représentent les bornes de la zone de recherche à chaque étape  
    left = 0  
    right = len(A) - 1  
    middle = None  
  
    while left <= right:  
        middle = (left + right) // 2 # Milieu de l'intervale [left, right]  
        if A[middle] < x:  
            left = middle + 1  
        elif A[middle] > x:  
            right = middle - 1  
        else:  
            return True  
  
    return False
```

## 1.3 Objectifs du cours

Pour expliquer l'objectif du cours, nous allons proposer une version mathématique du problème.

Soit  $f \in \mathcal{A}(\mathbb{N}, \{0, 1\})$  une fonction qui pour chaque indice  $i$  de  $A$ , indique si  $A[i]$  est strictement inférieur à  $a$ .

Si  $A = [1, 5, 8, 9, 9, 11, 13]$  et  $a = 11$ , voici les valeurs successives de  $f$  sur chaque indice :

0, 0, 0, 0, 0, 1, 1

Dans un premier temps, on remarque que la place potentielle de 11 est le tout premier 1.

Dans un second temps, on remarque que  $f$  est toujours croissante.

### Note

L'objectif du cours sera donc de retrouver une telle fonction, pour pouvoir appliquer la recherche dichotomique.

### Theorem 1.3.1 (Complexité)

La **recherche dichotomique** est toujours en  $\mathcal{O}(T(n) \log n)$ , où  $T(n)$  est la complexité pour calculer  $f(n)$ .

## **2. Plus grand / petit élément qui vérifie une condition**

---

## 2.1 Exemple - plus petit élément supérieur à $x$

Pour illustrer comment faire cela, nous allons étudier un exemple.

Soit  $A$  un tableau trié à  $n$  valeurs trié, on cherche le plus petit élément  $a$  de  $A$  qui vérifie  $x \leq a$ .

En d'autres termes, si on considère la fonction  $f$  qu'on a défini avant, on cherche le plus petit indice  $i$  qui vérifie  $f(i) = 1$ .

Cette fois-ci, nous tombons sur 2 cas lors de la recherche dichotomique :

- $A[\frac{n}{2}] < a$ , donc  $a$  peut uniquement être après (ou à) l'indice  $\frac{n}{2}$  (soit dans  $A[\frac{n}{2}, n]$ )
- $A[\frac{n}{2}] \geq a$ , donc  $a$  peut uniquement être avant l'indice  $\frac{n}{2}$  (soit dans  $A[1, \frac{n}{2}]$ )

### Note

Remarquez qu'on n'exclut pas l'indice  $\frac{n}{2}$  ! Ici on attend que la recherche n'aie plus de candidats pour retourner la borne supérieur de la zone de recherche.

## 2.2 Implémentation de l'exemple

```
def minsup_search(A : list[int], x : int):  
    # l et r représentent les bornes de la zone de recherche à chaque étape  
    # l et r sont exclus de la liste car il se peut que le premier élément qui vérifie la  
    # condition ne soit pas dans la liste  
    left = -1  
    right = len(A)  
    middle = None  
  
    while left + 1 < right:  
        middle = (left + right) // 2 # Milieu de l'intervale [left, right]  
        if A[middle] < x:  
            left = middle  
        else:  
            right = middle  
  
    return right
```

### Note

On peut faire la même chose pour le plus grand élément inférieur à  $x$ , il suffit de modifier l'égalité et de renvoyer la borne left ...

## 2.3 Exemple sur un problème CSES

Voici un cas concret avec ce problème : [CSES - Factory machines](#), prenez votre temps d'abord pour faire le lien entre ce dernier et la recherche dichotomique.

## 2.3 Exemple sur un problème CSES

Voici un cas concret avec ce problème : [CSES - Factory machines](#), prenez votre temps d'abord pour faire le lien entre ce dernier et la recherche dichotomique.

### ① Question préliminaire 1

Comment j'obtiens le plus grand nombre de produits qu'on peut créer en  $T$  secondes ?

## 2.3 Exemple sur un problème CSES

Voici un cas concret avec ce problème : [CSES - Factory machines](#), prenez votre temps d'abord pour faire le lien entre ce dernier et la recherche dichotomique.

### ① Question préliminaire 1

Comment j'obtiens le plus grand nombre de produits qu'on peut créer en  $T$  secondes ?

### ① Question préliminaire 2

Si  $f(t)$  indique si les  $t$  produits sont faisables en  $T$  secondes. Est-ce que  $f$  est croissante sur  $\mathbb{N}$  ?

L'implémentation est laissée au lecteur. 😊

### **3. Astuces**

---

### Note

Dans les problèmes un peu complexes, voici ce qui devrait vous mettre la puce à l'oreille pour savoir si vous pouvez faire une recherche dichotomique :

- On recherche la plus petite valeur vérifiant une condition.
- Les éléments sont ordonnés d'une certaine manière.

### Caution

La recherche dichotomique peut ne pas être pertinente, lorsque le calcul de  $f(n)$  a une grosse complexité.

## 4. Crédits

---

Auteurs et images : Bilel MAHDJOUR

Pour le **Bureau des technologies** de **Polytech Nice Sophia**

## 5. Sources

---

- [CSES problemset](#)
- [CP algorithms](#)
- [Codeforces ITMO courses - binary search](#)